OOP INTERVIEW QUESTIONS

For Web Developers (3 Years Experience)

Purpose: This question set is designed to assess a web developer's understanding of Object-Oriented Programming principles and their practical application in web development. It covers foundational concepts, design patterns, and real-world scenarios.

1. CORE OOP CONCEPTS

1. Explain the four pillars of OOP (Encapsulation, Inheritance, Polymorphism, Abstraction). Provide examples from your web development work.

2. What is the difference between a class and an object? How would you create a reusable class for a common web element?

3. Explain encapsulation. Why is it important to hide internal implementation details? Give a practical web development example.

4. What is inheritance? Describe a scenario where you would use inheritance in a web application. What are its benefits and drawbacks?

5. Explain polymorphism. How have you used it in your previous projects?

6. What is method overriding vs method overloading? Which is more relevant in JavaScript/your primary language?

7. What is abstraction? How does it help in designing web applications?

8. Explain the difference between composition and inheritance. When would you use one over the other?

2. ACCESS MODIFIERS & VISIBILITY

1. Explain public, private, and protected access modifiers. How do they enforce encapsulation?

2. In JavaScript, how do you implement private properties/methods? Show different approaches (# syntax, closures, WeakMap).

3. What is the purpose of getters and setters? Provide a practical example from your web projects.

4. What are static methods and properties? When would you use them in a web application?

5. Explain the concept of property visibility in different OOP languages you've used.

3. CLASSES & OBJECTS

1. Walk me through creating a class. What is a constructor and why is it important?

2. What is the difference between instance variables and class variables? Provide examples.

3. How do you properly initialize an object? What are best practices?

4. Explain the concept of 'this' in OOP. How does it behave differently in JavaScript vs other languages?

5. What is a destructor/finalizer? Do you use it in web development?

6. How do you prevent instantiation of a class? Why would you want to do this?

4. INHERITANCE & HIERARCHY

1. Describe a real inheritance hierarchy you've built for a web project. Why was it the right approach?

2. Explain the difference between single and multiple inheritance. How do languages handle multiple inheritance?

3. What is the super/parent keyword used for? Provide code examples.

4. What problems can deep inheritance hierarchies cause? How would you refactor them?

5. Explain the Liskov Substitution Principle (LSP). How does it relate to inheritance?

6. What is method resolution order (MRO)? How does it work in Python or your primary language?

7. Can you override a parent class's private method? Why or why not?

5. POLYMORPHISM & INTERFACES

1. Explain compile-time (static) vs runtime (dynamic) polymorphism. Which is more common in web development?

2. What is an interface? How does it differ from an abstract class?

3. Have you used interfaces in your web projects? Provide a specific example.

4. What is duck typing? How is it relevant to JavaScript developers?

5. Explain method overloading. If your language doesn't support it natively, how would you achieve similar functionality?

6. What is the Strategy pattern and how does it use polymorphism?

6. DESIGN PATTERNS

1. Explain the Singleton pattern. Where have you used it in web development? What are its drawbacks?

2. Describe the Factory pattern. How is it useful in creating objects?

3. Explain the Observer pattern. How is it used in web development (e.g., event listeners)?

4. What is the Decorator pattern? Provide a practical web development example.

5. Describe the MVC/MVVM architecture. How do OOP principles apply to it?

6. What is the Repository pattern? How does it help with data management?

7. Explain the Dependency Injection pattern. Why is it important?

8. Have you used the Builder pattern? When would you use it?

7. SOLID PRINCIPLES

1. **Explain the Single Responsibility Principle (SRP). Show how you apply it in your code.**

2. **What is the Open/Closed Principle? How does inheritance and polymorphism relate to it?**

3. **Explain the Liskov Substitution Principle (LSP). Give an example of violating it.**

4. **What is the Interface Segregation Principle (ISP)? How does it improve code design?**

5. **Explain the Dependency Inversion Principle (DIP). How do you apply it in web applications?**

6. **Give an example of code you've written that violates a SOLID principle and how you refactored it.**

8. ADVANCED OOP CONCEPTS

1. **Explain mixins and traits. How are they different from inheritance?**

2. **What is a prototype and prototypal inheritance? How does it compare to class-based inheritance?**

3. **Describe generics/templates. How have you used them in your web projects?**

4. **What is reflection/introspection? Have you used it in web development?**

5. **Explain the concept of metaclasses. Is it relevant to web development?**

6. **What are anonymous/lambda classes? When would you use them?**

9. REAL-WORLD SCENARIOS

1. Design a User authentication system using OOP principles. What classes would you create?

2. How would you design a shopping cart system? What classes and relationships would you use?

3. Describe how you would structure a blog application using OOP. What classes represent what entities?

4. Design a notification system (email, SMS, push notifications). How would you use polymorphism?

5. You have a codebase with tight coupling. How would you refactor it using OOP principles?

6. Design an API response handler that supports multiple formats (JSON, XML, CSV). How would you use OOP?

7. Create a class hierarchy for different types of users (Admin, Moderator, User). What challenges might arise?

8. How would you design a plugin system that allows extending functionality without modifying core code?

10. LANGUAGE-SPECIFIC QUESTIONS

1. Explain how JavaScript's prototypal inheritance differs from classical OOP. How do ES6 classes fit in?

2. In your primary language, how do you handle null/undefined object references safely?

3. Explain the difference between value types and reference types in your language.

4. How does garbage collection relate to OOP? Is it something you consider in design?

5. What are the differences between 'const' and 'let' in JavaScript? How do they relate to OOP principles?

6. How do you handle asynchronous operations in an OOP context? Show an example with callbacks/promises/async-await.

11. TESTING & MAINTAINABILITY

1. **How do OOP principles help with writing testable code?**
2. **What is a Mock object? How would you use it in testing?**
3. **How would you test a class with dependencies? Explain dependency injection in testing.**
4. **What is code coverage? How do you achieve high coverage with OOP code?**
5. **Explain refactoring. What OOP principles guide your refactoring decisions?**
6. **How do you identify and fix code smells in an OOP codebase?**

12. PERFORMANCE & BEST PRACTICES

1. **How can poor OOP design impact application performance?**
2. **What is lazy loading and how does it relate to OOP design?**
3. **Explain object pooling. When would you use it in web development?**
4. **How do you handle circular dependencies in your class design?**
5. **What are common mistakes when using inheritance?**
6. **How do you keep your classes lean and focused?**

EVALUATION NOTES

Scoring Tips:
• Look for practical examples from real projects, not just theoretical knowledge
• Assess ability to explain complex concepts in simple terms
• Evaluate problem-solving approach and design thinking
• Consider whether they've learned from mistakes and improved their practices
• Check for depth: distinguish between surface-level and deep understanding
• Assess code quality awareness (readability, maintainability, testability)

Red Flags:
• Using inheritance for everything
• Not understanding the purpose of OOP principles
• Ignoring SOLID principles
• Poor separation of concerns
• Difficulty explaining their own code decisions
• No awareness of design patterns

Green Flags:
• Clear understanding of when to use each OOP concept
• Examples of refactoring poor OOP design
• Awareness of trade-offs and context-dependent decisions
• Continuous learning mindset
• Ability to apply SOLID principles in practice

Generated on March 01, 2026